Documentation for Google Drive

Group members

Group ID: 16
2019090 Rishi Raj
2019118 Tanmay Rajore
2019450 Sushant Gupta

Google Drive is a file storage and synchronization service developed by Google. It allows users to store files in the cloud (on Google's servers), synchronize files across devices, and share files. Google Drive encompasses Google Docs, Google Sheets, and Google Slides, which are a part of the Google Docs Editors office suite that permits collaborative editing of documents, spreadsheets, presentations, drawings, forms, and more. Files created and edited through the Google Docs suite are saved in Google Drive.

Application features

1. **Scalability** - scalability in cloud computing refers to the ability to increase or decrease IT resources as needed to meet changing demand.
2. **Elasticity** - elasticity is defined as "the degree to which a system is able to adapt to workload changes by provisioning resources automatically, such that at each point in time the available resources match the current demand as closely as possible.
3. **Failover** - backup operational mode that automatically switches to a standby database, server or network if the primary system fails, or is shut down for servicing
4. **Sharing** - Google Drive enables sharing of files between google suite users and non google suite users by the way of unique link or by access control in the Google suite itself.
5. **Security** - In a file-sharing service, the privacy and security of user data are essential. To handle this, all files uploaded to Google Drive are stored in secure, encrypted data centers.
6. **Usability and accessibility** - It has an easy-to-use user interface and provide a feature of drag and drop. If you have a good internet connection, you can access your files, which are saved online somewhere in the data centers.
7. **User Interface** - It has a good, easy to use interface with a lot of elements.
    a. Most recent documents on top of the page
    b. a list of all of your folders just below
    c. a simple navigation on the left which can be used to
        i. Create a new document
        ii. View shared folders between computers
        iii. View documents shared outside your personal drive
        iv. See recent, starred, or deleted documents

API/Function

Authentication

POST http://localhost:8000/auth-token/ HTTP/1.1

```
{
   "username":"admin",
   "password":"password"
}
```

Description

Body parameters:

- Username - username of the user
- Password - password of the user

This API is used to authenticate the user from the frontend. An JWT token is return as response for use in authentication of future requests either by the frontend or the user itself.

GET http://localhost:8000/api/users/me/ HTTP/1.1
Authorization: Token {{auth_token}}

Description

This API is used to get the details of the user. The response include the username, name, email address etc. It is also used to verify that the token is still valid.

GET http://localhost:8000/users/{{username}}/ HTTP/1.1
Authorization: Token {{auth_token}}

Description

This API is used by the admin website to fetch details of any user

GET http://localhost:8000/users/ HTTP/1.1
Authorization: Token {{auth_token}}

Description

This API is used by the admin to get a list of users.

PATCH http://localhost:8000/users/ HTTP/1.1
Authorization: Token {{auth_token}}

Description

This API is used by the admin to edit the details of any user details (except username and password).

File

POST http://localhost:8000/api/files/ HTTP/1.1
Authorization: Token {{auth_token}}
{
"Name”: string,
"Folder: string"
“File” : file
}
Description
Body parameters:

- Name - Name of the file
- Folder - The owner/parent folder of the file.

This API is used to create a new file.

GET http://localhost:8000/api/files/{{id}} HTTP/1.1
Authorization: Token {{auth_token}}
Description
Retrieve the details about the file.

PATCH http://localhost:8000/api/files/{{id}} HTTP/1.1
Authorization: Token {{auth_token}}
{
"Name”: string,
"Folder: string"
“File” : file
}
Description
Used to update/patch the details about the file.

DELETE http://localhost:8000/api/files/ HTTP/1.1
Authorization: Token {{auth_token}}
Description
This API is used to delete the file.

Folder

POST http://localhost:8000/api/folder/ HTTP/1.1
Authorization: Token {{auth_token}}
{
"Name”: string,

```
   "Folder: string"
}
```
Description
Body parameters:
- Name - Name of the file
- Folder - The owner/parent folder of the current folder.

This API is used to create a new file.

```
GET http://localhost:8000/api/folders/{{id}} HTTP/1.1
Authorization: Token {{auth_token}}
```
Description
Retrieve the details about the folder.

```
PATCH http://localhost:8000/api/folders/{{id}} HTTP/1.1
Authorization: Token {{auth_token}}
{
   "Name”: string,
   "Folder: string"
   “File” : file
}
```
Description
Used to update/patch the details about the folder.

```
DELETE http://localhost:8000/api/folders/ HTTP/1.1
Authorization: Token {{auth_token}}
```
Description
This API is used to delete the folder.

Sharing the folder/file

```
POST http://localhost:8000/api/share/folder/ HTTP/1.1
Authorization: Token {{auth_token}}
{
   "type”: string (limited choices),
   “folder”: int
   “user” : string
}
```
Description
Body parameters:
- Type - There are 2 types of access provided while sharing. Editor and Viewer.
- folder - The unique_id for the folder to be shared.
- User - The username of the user to whom file shall be shared.

This API shares the folder with other users.

POST http://localhost:8000/api/share/file/ HTTP/1.1
Authorization: Token {{auth_token}}
{
"type": string (limited choices),
"file": int
"user" : string
}

Description

Body parameters:

- Type - There are 2 types of access provided while sharing. Editor and Viewer.
- File - The unique_id for the file to be shared.
- User - The username of the user to whom file shall be shared.

This API share the file with other users.

GET http://localhost:8000/api/utility/get_entities/{folder_id}
Authorization: Token {{auth_token}}

Description
The API returns all the files/folders within the folder.

GET http://localhost:8000/api/utility/access_type//{entity_id}?type={entity_type}

Description

This API is used to get the access type of an entity (file or folder) given its ID. It recursively goes up to the root folder and on the way checks if any folder has viewer/editor access. We need to do this because if we share a folder, all the files that are present in the folder or its descendants are also shared.

Authorization: Token {{auth_token}}

Description
The API returns all the files/folders within the folder.

Our Approach: SaaS (Software as a Service)

Cloud Services Used: Amazon Beanstalk Elastic(EC2,S3) (PaaS)

Reason: We require storage, computing, and monitoring so the best approach to build our SaaS product was to use the Amazon Beanstalk Elastic service

AWS cloudFormation -> Allows for automatic scaling of required compute, storage
Amazon EC2 instances -> Provides Compute Resources for the application
S3 Buckets -> provides storage for the application
Amazon CloudWatch -> provides monitoring and analytics

Source code documentation

Code structure:

Drive

Subfolders	Description
Drive_backend	Contains the source code files for backend operations on Django stack
Frontend	Contains the source code files for frontend operations on React Native tech stack

Drive_backend
NOTE: certain application dependent autogenerated files are excluded

subfolders	Description
config	Configuration settings for Django web app
setup.cfg	Setup file for initializing the Django web app
drive_backend	Contains -file_system
utility	Scripts for installation of dependencies

Frontend:
NOTE: certain auto generated files are excluded.(setuptest.js,app.test.js etc)

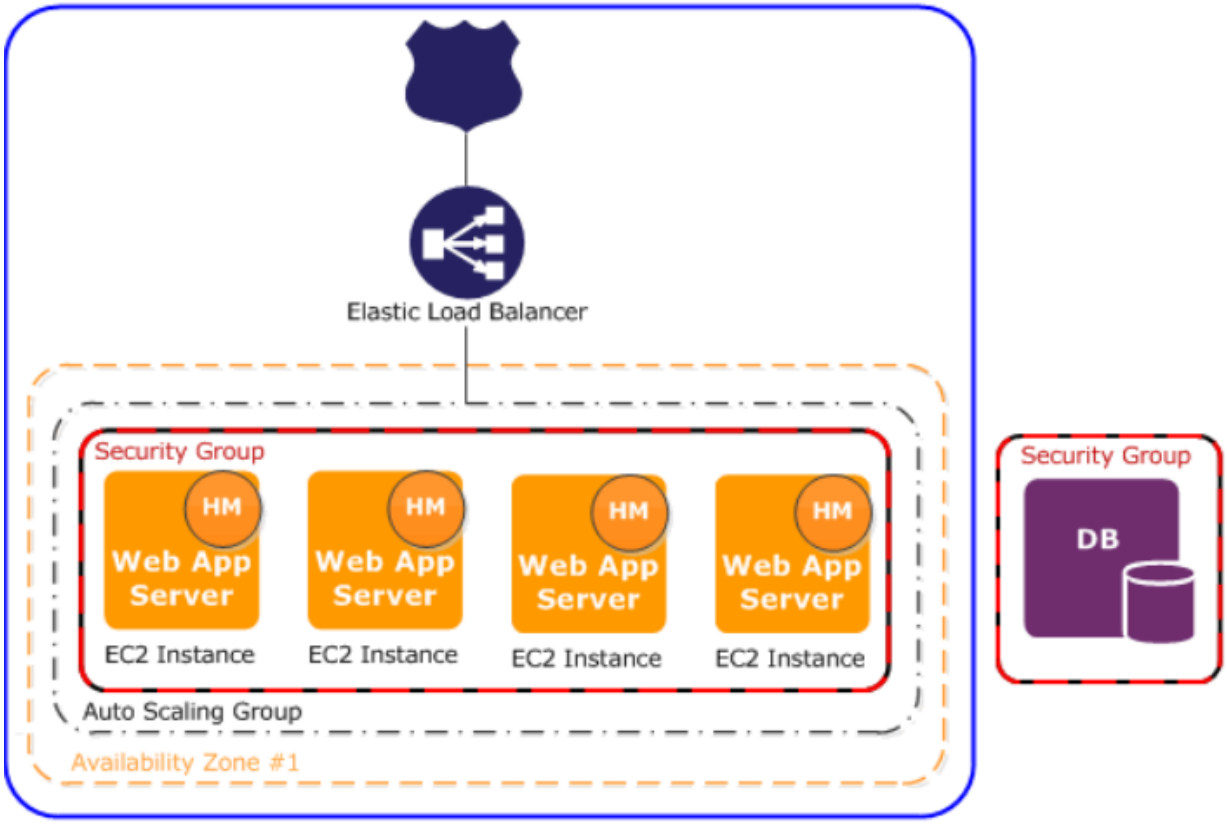
subfolders	Description
public	Contains predefined pages like robots.txt,index.html
src	Source code for the frontend written in react - Components - Contains the .js files required. - Styles - App.js

Scalability and Elasticity:

Using amazon Auto scaling tool for EC2 instances and S3 storage and RDBMS-based storage, we ensure that our application is scalable and Elastic in scaling up and scaling down as required.

Availability:

Since Amazon in its SLA states the availability of the products such as EC2 instances, S3 and RDBMS based to be 99.99% available, our application will meet the requirement for availability.



Database structure:

The following section explains the database structure which is implemented at the backend level for managing the files and folder structure created by the user using the Frontend UI.

The Folder schema:
Name: stores the Name of the folder (max limit of 100)
Owner: the userid unique token
Parent : the parent folder/directory.
Users: the list of other user id which have access to the folder.

```
class Folder(models.Model):
    name = models.CharField(max_length=100, blank=False, default='Untitled')
    owner = models.ForeignKey(User, on_delete=models.CASCADE, null=True, default=None)
    parent = models.ForeignKey('self', on_delete=models.CASCADE, null=True, default=None)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
    users = models.ManyToManyField(User, through='FolderAccess', related_name='folders', related_query_name='folder')
    class Meta:
        constraints = [
            models.UniqueConstraint(fields=['name','parent'], name='folder_parent_unique_constraint')
        ]
```

The File schema:

```
class File(models.Model):
    name = models.CharField(max_length=100, blank=False, default='Untitled')
    file = models.FileField()
    owner = models.ForeignKey(User, on_delete=models.CASCADE, null=True, default=None)
    folder = models.ForeignKey(Folder, on_delete=models.CASCADE, null=True, default=None)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
    users = models.ManyToManyField(User, related_name='files', related_query_name='file', through='FileAccess')
    class Meta:
        constraints = [
            models.UniqueConstraint(fields=['name','folder'], name='name_folder_unique_constraint')
        ]
```

The file Access schema

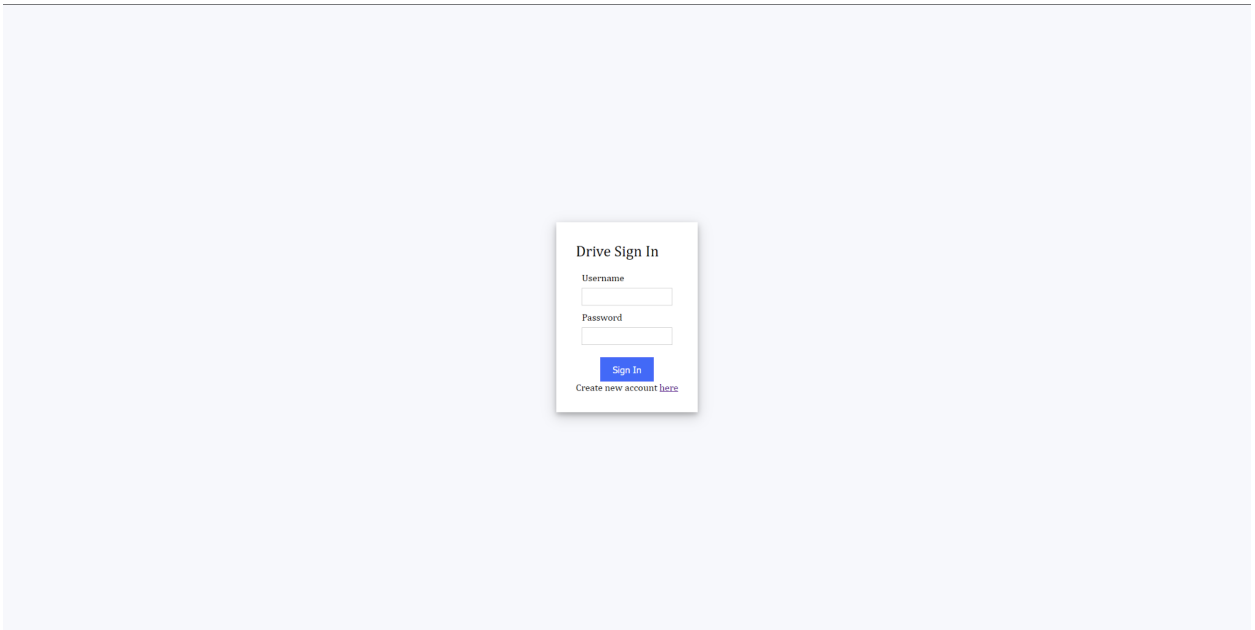
```
class FileAccess(models.Model):
    file = models.ForeignKey(File, on_delete=models.CASCADE, null=True)
    user = models.ForeignKey(User, on_delete=models.CASCADE, null=True)
    type = models.CharField(max_length=10, choices=[('viewer', 'viewer'), ('editor', 'editor')], default='viewer')
    updated = models.DateTimeField(auto_now=True)
    created = models.DateTimeField(auto_now_add=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(fields=['file', 'user'], name='file_user_unique_constraint')
        ]
```

The folder Access schema

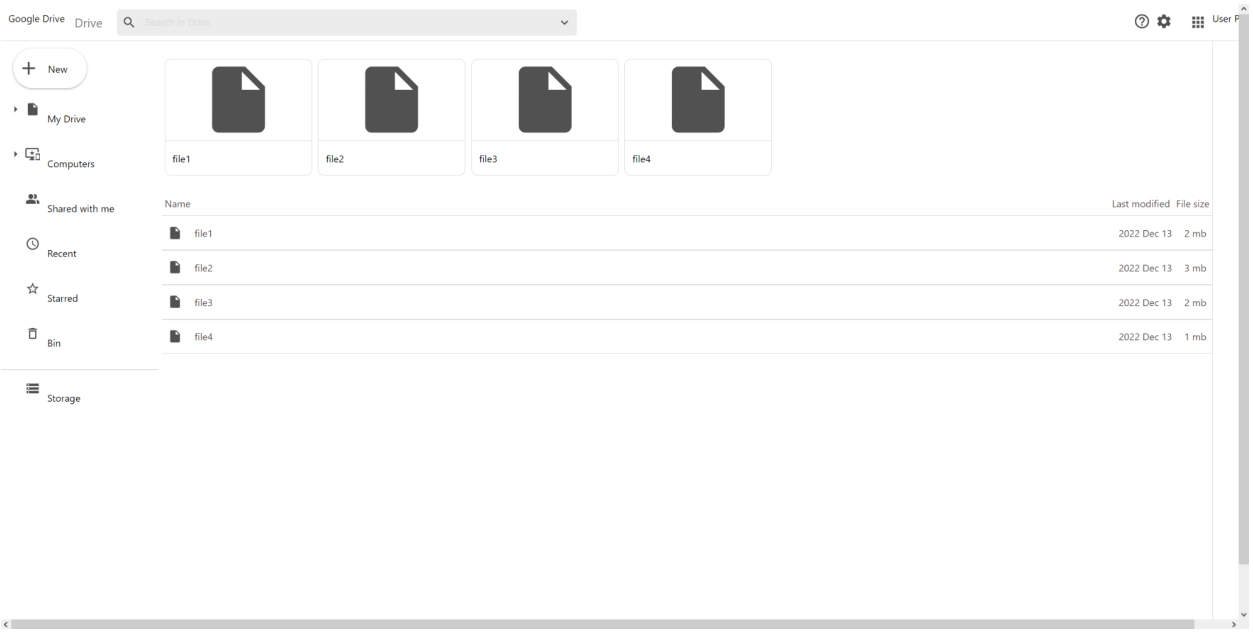
```
class FolderAccess(models.Model):
    folder = models.ForeignKey(Folder, on_delete=models.CASCADE, null=True)
    user = models.ForeignKey(User, on_delete=models.CASCADE, null=True)
    type = models.CharField(max_length=10, choices=[('viewer', 'viewer'), ('editor', 'editor')], default='viewer')
    updated = models.DateTimeField(auto_now=True)
    created = models.DateTimeField(auto_now_add=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(fields=['folder', 'user'], name='folder_user_unique_constraint')
        ]
```

Frontend UI:

Login Screen:



Main Dashboard screen:



Frontend source code structure:
UI components:
Header - implemented using header.js

```
import SearchIcon from '@material-ui/icons/Search';
import ExpandMoreIcon from '@material-ui/icons/ExpandMore';
import HelpOutlineIcon from '@material-ui/icons/HelpOutline';
import SettingsIcon from '@material-ui/icons/Settings';
import AccountCircleIcon from '@material-ui/icons/AccountCircle';
import AppsIcon from '@material-ui/icons/Apps';

const Header = ({ userPhoto }) => {
    return (
        <div className='header'>
            <div className="header__logo">
                <img src={"GDriveLogo"} alt="Google Drive" />
                <span>Drive</span>
            </div>
            <div className="header__searchContainer">
                <div className="header__searchBar">
                    <SearchIcon />
                    <input type="text" placeholder='Search in Drive' />
```

The code firstly imports the assets like icons and geometrical shapes which are to be rendered. Then using different divisions the appropriate logo is placed.

New File - implemented using newfile.js

```
function getModalStyle() {
    return {
        top: `50%`,
        left: `50%`,
        transform: `translate(-50%, -50%)`,
    };
}

const useStyles = makeStyles((theme) => ({
    paper: {
        position: 'absolute',
        width: 400,
        backgroundColor: theme.palette.background.paper,
        border: '2px solid #000',
        boxShadow: theme.shadows[5],
        padding: theme.spacing(2, 4, 3),
    },
}));


const NewFile = () => {
    const classes = useStyles();

    const [modalStyle] = useState(getModalStyle);
    const [open, setOpen] = useState(false);
    const [file, setFile] = useState(null)
    const [uploading, setUploading] = useState(false)
```

The code here defines the new file icon and its position,background color etc and then using appropriate data the required states for the text fields are set.

Other .js files and its functions are as follows:

File	Description
sidebar.js	Used to render the side bar and its features

File	Description
sideicon.js	Used for rendering the icons like mydrive,bin etc on the sidebar
filescard.js	Used for rendering the file icon for the uploaded files
fileitem.js	Used for file item view of the uploaded files
showfiles.js	Used for fetching files from the backend and then rendering the information and details about the files.